

Immobil'IA — Database Architecture Proposal

Prototype scope · self-hosted / Docker · generated from codebase analysis (src/types, src/services)

Answer up front: 3 databases + 1 object store, all self-hosted in Docker. **PostgreSQL** (relational core), **Qdrant** (vector search: semantic listing search, colocation matching, image dedup/fraud, legal-doc RAG), **Redis** (cache, sessions, real-time chat/notifications), **MinIO** (S3-compatible file storage for photos/videos/floor plans). No single database covers all of this alone — relational integrity (bookings, offers, messages) and vector similarity (AI features) are different problems, so a polyglot but minimal stack was chosen over forcing everything into one engine or over-engineering with 5+ services.

1. Why 4 services, not 1 or 6

The app's own mock services (ai-service.ts, investor-service.ts) already assume four categories of data that don't fit one storage model well:

- **Structured, relational data** — users, properties, offers, messages, reviews: needs joins, transactions, constraints. → PostgreSQL.
- **Similarity search over embeddings** — "recherche hybride" natural-language property search, colocation compatibility, reverse-image fraud detection, legal-text retrieval for contract generation: needs approximate nearest-neighbor search, not SQL. → Qdrant.
- **Ephemeral / high-churn data** — sessions, rate limits on AI endpoints, chat presence, cached AI results (price estimates, investor reports): needs sub-millisecond reads, TTL expiry, pub/sub. → Redis.
- **Binary files** — property photos, videos, virtual tours, floor plans, avatars: never belongs in a DB row. → MinIO (S3 API, drop-in swap for AWS S3 later).

A single Postgres+pgvector could technically merge cases 1 and 2, but since all four AI-heavy features (search, colocation, image fraud, legal RAG) are planned for *real* implementation — not just mocked — a dedicated vector engine (Qdrant) gives better ANN performance and metadata filtering without complicating the relational schema.

PostgreSQL 16 core

Users, properties, offers, messages, reviews, investor reports, notifications — everything transactional.

Qdrant vector

4 collections: property search, colocation matching, image embeddings, legal-doc chunks.

Redis 7 cache

Sessions, AI-result cache, rate limiting, chat pub/sub, notification fanout.

MinIO files

Photos, videos, virtual tours, floor plans, avatars. S3-compatible.

2. PostgreSQL schema

2.1 Identity

Table	Columns	Notes
users	id uuid, first_name, last_name, email unique, password_hash, avatar_url → minio key, role enum(vendeur acheteur investisseur), member_since date, verified bool, rating numeric, review_count int	role drives which dashboard (vendeur/investisseur space) the user sees

2.2 Properties & location

Table	Columns	Notes
properties	id , owner_id -users, title, description, type enum(appartement maison studio loft duplex immeuble), transaction enum(vente location), status enum(disponible sous_offre loue vendu brouillon en_verification), price numeric, charges numeric, surface numeric, rooms int, bedrooms int, bathrooms int, floor int, year_built int, energy_class enum(A..G), address, city, postal_code, neighborhood_id -neighborhoods, position_x, position_y, source enum(plateforme web), source_name, published_at timestampz, quality_score int, honesty_score int, ai_price_estimate numeric, view_count int, favorite_count int, availability_verified bool	quality_score / honesty_score / ai_price_estimate are written by the AI service, not user input
property_media	id , property_id -properties, kind enum(photo video virtual_tour floor_plan), storage_key → minio, position int	replaces the images[]/videoUrl/virtualTourUrl/floorPlanUrl fields with one table
property_features	property_id -properties, feature text	or a text[] column directly on properties — fine at this scale
favorites	user_id -users, property_id -properties, created_at	composite PK (user_id, property_id)

Table	Columns	Notes
neighborhoods	id , name, city, score_overall, score_securite, score_transport, score_bruit, score_commerces, score_ecoles, flagged bool, flag_reason, trend enum(hausse stable baisse), price_per_sqm numeric	
points_of_interest	id , neighborhood_id -neighborhoods, kind enum(ecole universite transport commerce parc hopital bruit pollution embouteillage), label, position_x, position_y, distance_min int, negative bool	

2.3 Transactions & social proof

Table	Columns	Notes
offers	id , property_id -properties, buyer_id -users, amount numeric, status enum(en_attente acceptee refusee expiree), date timestampz	
reviews	id , property_id -properties, author_id -users, rating int, comment text, date timestampz	
abuse_reports	id , property_id -properties, reporter_id -users, kind enum(scam comportement), reason text, status enum(en_cours traite), date	
competitor_listings	id , property_id -properties (ref. property), title, price, surface, price_per_sqm, distance_km, image_url, source enum(plateforme web), published_at	cache of scraped comparables (EF-V-04); short TTL, could live in Redis instead if refreshed often

2.4 Messaging & notifications

Table	Columns	Notes
conversations	id , property_id -properties, last_message text, last_message_at timestampz	
conversation_participants	conversation_id -conversations, user_id -users, unread_count int	composite PK
messages	id , conversation_id -conversations, sender_id -users, content text, sent_at timestampz, read bool	live delivery goes through Redis pub/sub; Postgres is the durable log
notifications	id , user_id -users, kind enum(offre_similaire message prix visite systeme alerte), title, body, date, read bool, href	
search_history	id , user_id -users, query text, date, result_count int, alert_enabled bool	

2.5 Investor module

Table	Columns	Notes
investment_reports	id , property_id -properties, generated_at, gross_yield, net_yield, monthly_rent, annual_costs, cashflow, payback_years, risk_score int, forecast_now, forecast_5y, forecast_10y, annual_growth_percent, summary text	regenerated on demand; consider caching latest in Redis with short TTL
risk_factors	report_id -investment_reports, label, level enum(faible modere eleve), detail text	
what_if_scenarios	report_id -investment_reports, label, description, delta_yield, delta_value	

2.6 Colocation & tools

Table	Columns	Notes
coloc_profiles	id , user_id -users, age int, occupation, budget numeric, city, tags text[], bio text	compatibility score is <i>computed</i> via Qdrant, not stored here
decoration_ideas / repair_findings / staging_examples	standard content tables; repair_findings has property_id -properties, zone, issue, severity enum, estimated_cost_min/max, score_impact, price_impact	mostly reference/seed content for the "outils" pages
generated_contracts	id , property_id -properties, reference, content jsonb (sections/regulations), verified bool, verification_issues text[], created_at	content stored as jsonb since sections are variable-length structured text

3. Qdrant — vector collections

Four collections, one per AI feature that needs similarity search. Each stores a vector plus a payload (metadata) used for filtered search — Qdrant supports pre/post-filtering on payload fields alongside the ANN search, which is what makes it a better fit here than raw pgvector for filtered, faceted search (e.g. "similar 2-bedroom apartments in Lyon under 300k").

Collection	Vector source	Payload (filterable)	Used by
property_embeddings	Embedding of title + description + features + city	property_id, city, type, transaction, price, status	Natural-language property search (searchProperties "recherche hybride"), similar-listings
coloc_embeddings	Embedding of bio + tags + occupation	profile_id, city, budget, age	Colocation compatibility matching (colocProfiles)
image_embeddings	CLIP-style image embedding per photo	property_id, storage_key, source (plateforme web), portal_name	Reverse-image fraud/dup detection across portals (verifyAvailability, "recherche par image inversée")
legal_doc_chunks	Embedding of chunked legal text (loi 6 juillet 1989, Code pénal, Code de la consommation, CCH)	law_reference, article, transaction_type	RAG grounding for contract generation & regulation checks (generateContract, detectFraud)

4. Redis — usage patterns

Pattern	Key shape	TTL
Session / auth token	session:{token}	7d
AI result cache (price estimate, investor report)	ai:price:{property_id}	1h
Rate limiting AI endpoints	ratelimit:{user_id}:{endpoint}	1m sliding window
Chat real-time delivery	pub/sub channel chat:{conversation_id}	n/a
Notification fanout	pub/sub channel notif:{user_id}	n/a
Popular search result cache	search:{filters_hash}	5m

5. MinIO — buckets

- property-photos / property-videos / floor-plans — linked from property_media.storage_key
- avatars — users & coloc profiles
- tool-assets — decoration/staging before-after images

6. Example data flow — semantic property search

- User types free-text query in /recherche
- API embeds the query (same model used to index properties)
- API queries Qdrant "property_embeddings" – top-K nearest vectors, filtered by payload (city/transaction/price if set)
- API takes the returned property_ids, fetches full rows from PostgreSQL "properties" (price, honesty_score, etc.)
- Result merged, sorted by honesty_score (as current mock does), response cached in Redis under search:{filters_hash} for 5 min
- Property photos resolved to MinIO URLs client-side

7. Docker Compose sketch

Service	Image	Port
postgres	postgres:16	5432
qdrant	qdrant/qdrant	6333
redis	redis:7-alpine	6379
minio	minio/minio	9000 / 9001 (console)

Generated by exploring `src/types/index.ts` and `src/services/*.ts` of the `immobilia_frontend` repo (mock layer only — no backend exists yet). Enum values and field names mirror the TypeScript interfaces directly so the eventual API/ORM layer maps 1:1 onto this schema.